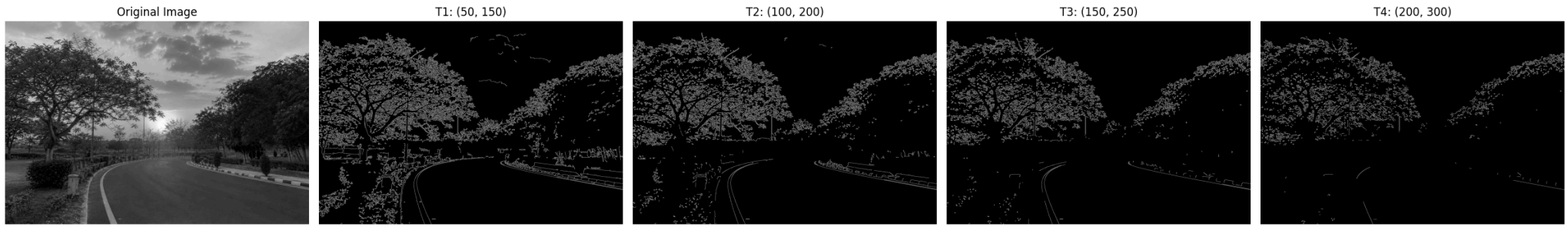
```
# ============================================================
# THRESHOLD EFFECT IN EDGE DETECTION
# Correct Google Colab Program
# ============================================================

# Import libraries
import cv2
import matplotlib.pyplot as plt
from google.colab import files

# ------------------------------------------------------------
# Step 1: Upload an image
# ------------------------------------------------------------
uploaded = files.upload()

# Get uploaded file name
image_path = next(iter(uploaded))

# ------------------------------------------------------------
# Step 2: Read the image
# ------------------------------------------------------------
img = cv2.imread(image_path)

# Check whether image is loaded correctly
if img is None:
    raise ValueError("Image not loaded. Please upload a valid image.")

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# ------------------------------------------------------------
# Step 3: Reduce noise using Gaussian Blur
# ------------------------------------------------------------
blur = cv2.GaussianBlur(gray, (5, 5), 0)

# ------------------------------------------------------------
# Step 4: Apply Canny Edge Detection with different thresholds
# ------------------------------------------------------------
thresholds = [
    (50, 150),
    (100, 200),
    (150, 250),
    (200, 300)
]

edge_images = []

for low, high in thresholds:
    edges = cv2.Canny(blur, low, high)
    edge_images.append((low, high, edges))
```

```
# ------------------------------------------------------------
# Step 5: Display results in horizontal format
# ------------------------------------------------------------
fig, axes = plt.subplots(1, 5, figsize=(24, 5))

# Original grayscale image
axes[0].imshow(gray, cmap='gray')
axes[0].set_title("Original Image", fontsize=12)
axes[0].axis('off')

# Edge detection results
for i, (low, high, edge) in enumerate(edge_images):
    axes[i + 1].imshow(edge, cmap='gray')
    axes[i + 1].set_title(f"T{i+1}: ({low}, {high})", fontsize=12)
    axes[i + 1].axis('off')

# Main title
plt.suptitle("Threshold Effect in Edge Detection",
             fontsize=18, fontweight='bold')

plt.tight_layout()
plt.show()

# ------------------------------------------------------------
# Step 6: Save the output image
# ------------------------------------------------------------
fig.savefig("threshold_effect_output.png",
            dpi=300, bbox_inches='tight')

print("Output saved as: threshold_effect_output.png")
```

Choose files img6.png

img6.png(image/png) - 3045754 bytes, last modified: 10/08/2026 - 100% done

Saving img6.png to img6.png



Output saved as: threshold_effect_output.png